

Laboratorio I

a.a. 2025/2026

TypeScript II

Controllo dei Tipi

- Il type system di TypeScript è di tipo strutturale (*Structural Type System*)
 1. Il controllo e la comparazione dei tipi di dato da parte del compilatore si basa sulla struttura dei dati, sulla loro forma
 2. Chiamato anche “*duck typing*” o “*structural subtyping*”

```
function printLabel(labeledObj: { label: string }) {  
  console.log(labeledObj.label);  
}
```

L'oggetto passato deve avere una proprietà **label** di tipo **string**

```
let myObj = { size: 10, label: "Size 10 Object" };  
printLabel(myObj);
```

myObj ha più proprietà, ma questo non importa: deve esistere almeno quella che **printLabel()** si aspetta!

Controllo dei Tipi

- Il type system di TypeScript è di tipo strutturale (*Structural Type System*)
 1. Il controllo e la comparazione dei tipi di dato da parte del compilatore si basa sulla struttura dei dati, sulla loro forma
 2. Chiamato anche “*duck typing*” o “*structural subtyping*”

```
function printLabel(labeledObj: { label: string }) {  
  console.log(labeledObj.label);  
}
```

L'oggetto passato deve avere una proprietà **label** di tipo **string**

```
let myObj = { size: 10, label: 33 };  
printLabel(myObj);
```

Questo darebbe **errore** (**label** non è **string** -- sarebbe lo stesso se **label** non esistesse)

Interfacce

- TypeScript consente la definizione di una **interfaccia** per definire la **struttura** di un oggetto

```
interface LabeledValue {  
  label: string;  
}
```

```
function printLabel(labeledObj: LabeledValue) {  
  console.log(labeledObj.label);  
}
```

```
let myObj = { size: 10, label: "Size 10 Object" };  
printLabel(myObj);
```

Interfacce

- TypeScript consente la definizione di una **interfaccia** per definire la **struttura** di un oggetto

```
interface LabeledValue {  
  label: string;  
}
```

Non dobbiamo specificare che l'oggetto che passiamo a **printLabel** implementa l'interfaccia (come in altri linguaggi). In TS è solo la forma che conta!

```
function printLabel(labeledObj: LabeledValue) {  
  console.log(labeledObj.label);  
}
```

```
let myObj = { size: 10, label: "Size 10 Object" };  
printLabel(myObj);
```

Interfacce - Estensione

- Le interfacce possono essere estese (come possiamo fare con le classi)

```
interface IPerson {  
    name: string;  
    surname: string;  
}
```

```
interface IEmployee extends IPerson {  
    empCode: number;  
}
```

```
let empObj:IEmployee = { empCode:1, name:"Bill", surname:"White" }
```

Interfacce - Implementazione

- Come in altri linguaggi, è possibile forzare una classe a utilizzare il *contratto* definito da una interfaccia

```
interface ClockInterface {  
  currentTime: Date;  
}
```

```
class Clock implements ClockInterface {  
  currentTime: Date = new Date();  
  constructor(h: number, m: number) {}  
}
```

Interfacce - Implementazione

- Come in altri linguaggi, è possibile forzare una classe a utilizzare il *contratto* definito da una interfaccia

TS

```
1 interface ClockInterface {  
2   currentTime: Date;  
3 }  
4  
5 class Clock implements ClockInterface {  
6   currentTime: Date = new Date();  
7   constructor(h: number, m: number) {}  
8 }
```

JS

```
"use strict";  
class Clock {  
  constructor(h, m) {  
    this.currentTime = new Date();  
  }  
}
```


Interfacce - Implementazione

- Come in altri linguaggi, è possibile forzare una classe a utilizzare il *contratto* definito da una interfaccia
→ Non solo con proprietà, ma anche con metodi

```
interface ClockInterface {  
  currentTime: Date;  
  setTime(d: Date): void;  
}
```

```
class Clock implements ClockInterface {  
  currentTime: Date = new Date();  
  setTime(d: Date) {  
    this.currentTime = d;  
  }  
  constructor(h: number, m: number) {}  
}
```

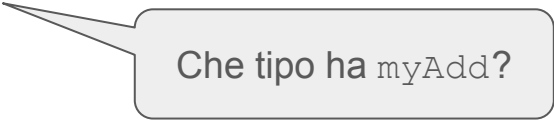
Il tipo di una funzione

- Come sappiamo, in JS le funzioni sono valori
- Quindi hanno un tipo!
- Cosa vuol dire questo in TS?

Il tipo di una funzione

- Come sappiamo, in JS le funzioni sono valori
- Quindi hanno un tipo!
- Cosa vuol dire questo in TS?

```
let myAdd = function (x: number, y: number): number {  
  return x + y;  
};
```



Che tipo ha `myAdd`?

Il tipo di una funzione

- Come sappiamo, in JS le funzioni sono valori
- Quindi hanno un tipo!
- Cosa vuol dire questo in TS?

```
let myAdd = function (x: number, y: number): number {  
  return x + y;  
};
```

- La segnatura della funzione!

```
let myAdd: (x: number, y: number) => number = function (x: number,  
  y: number): number {  
  return x + y;  
};
```

Il tipo di una funzione

- Come sappiamo, in JS le funzioni sono valori
- Quindi hanno un tipo!
- Cosa vuol dire questo in TS?

I nomi degli argomenti nel tipo funzione non devono essere uguali a quelli usati nella funzione (ma ne aumenta la leggibilità)

- La segnatura della funzione!

```
let myAdd: (x: number, y: number) => number = function (x: number,  
  y: number): number {  
  return x + y;  
};
```

Il tipo di una funzione

- Ricordate sempre la type inference: possiamo lasciare i tipi anche solo su un lato della dichiarazione, e TS fa il resto!

```
// 'x' e 'y' hanno il tipo number
```

```
let myAdd = function (x: number, y: number): number {  
  return x + y;  
};
```

```
// myAdd2 ha il tipo funzione completo
```

```
let myAdd2: (baseValue: number, increment: number) => number = function (x, y) {  
  return x + y;  
};
```

Parametri opzionali di una funzione

- In TS tutti i parametri attesi da una funzione sono richiesti al momento della chiamata
- In JS non è così

Parametri opzionali di una funzione

- In TS tutti i parametri attesi da una funzione sono richiesti al momento della chiamata
- In JS non è così

JS

- Se mancano valgono **undefined**
- Se sono troppi quelli in eccesso vengono ignorati
- In ogni caso, tutti i parametri attuali forniti sono accessibili dalla funzione tramite la variabile locale implicita **arguments** (che è un array)

Parametri opzionali di una funzione

- In TS tutti i parametri attesi da una funzione sono richiesti al momento della chiamata
- In JS non è così

JS

- Se mancano valgono **undefined**
- Se sono troppi quelli in eccesso vengono ignorati
- In ogni caso, tutti i parametri attuali forniti sono accessibili dalla funzione tramite la variabile locale implicita **arguments** (che è un array)

```
function testParam(a,b,c) {  
  return a + " " + b + " " + c + " " + arguments[3];  
}  
console.log(testParam(1,2,3));           //→ 1 2 3 undefined  
console.log(testParam(1));               //→ 1 undefined undefined undefined  
console.log(testParam(1,2,3,4,5,6));     //→ 1 2 3 4
```

Parametri opzionali di una funzione

- In TS tutti i parametri attesi da una funzione sono richiesti al momento della chiamata
- In JS non è così (tutti opzionali, e se mancano diventano **undefined**)
- TS permette questo comportamento usando ?

```
function buildName(firstName: string, lastName?: string) {  
  if (lastName) return firstName + " " + lastName;  
  else return firstName;  
}
```

```
let result1 = buildName("Bob"); // OK  
let result2 = buildName("Bob", "Adams", "Sr."); // KO, troppi parametri  
let result3 = buildName("Bob", "Adams"); // OK
```

Parametri opzionali di una funzione

- In TS tutti i parametri attesi da una funzione sono richiesti al momento della chiamata
- In JS non è così (tutti opzionali, e se mancano diventano **undefined**)
- TS permette questo comportamento usando ?

```
function buildName(firstName: string, lastName?: string) {  
  if (lastName) return firstName + " " + lastName;  
  else return firstName;  
}
```

I parametri opzionali sempre **dopo** quelli obbligatori

```
let result1 = buildName("Bob"); // OK  
let result2 = buildName("Bob", "Adams", "Sr."); // KO, troppi parametri  
let result3 = buildName("Bob", "Adams"); // OK
```

Parametri opzionali di una funzione

- In TS tutti i parametri attesi da una funzione sono richiesti al momento della chiamata
- In JS non è così (tutti opzionali, e se mancano diventano **undefined**)
- TS permette questo comportamento usando ?

```
function buildName(firstName: string, lastName?: string) {  
  if (lastName) return firstName + " " + lastName;  
  else return firstName;  
}
```

tipo: (firstName: string, lastName?: string|undefined): string

Parametri opzionali di una funzione

- Inoltre è possibile fornire valori di inizializzazione ai parametri
 - Usati se il parametro non viene fornito o se `undefined`
- Quelli con *inizializzazione* che vengono dopo tutti quelli richiesti sono trattati come *opzionali* (e quindi possono essere omessi in chiamata)
 - In questo caso, la funzione ha lo stesso tipo di quella con parametro opzionale

Parametri opzionali di una funzione

- Inoltre è possibile fornire valori di inizializzazione ai parametri
 - Usati se il parametro non viene fornito o se `undefined`
- Quelli con *inizializzazione* che vengono dopo tutti quelli richiesti sono trattati come *opzionali* (e quindi possono essere omessi in chiamata)
 - In questo caso, la funzione ha lo stesso tipo di quella con parametro opzionale

```
function buildName(firstName: string, lastName = "Woman") {  
    if (lastName) return firstName + " " + lastName;  
    else return firstName;  
}  
  
let result1 = buildName("Wonder"); // "Wonder Woman"  
let result2 = buildName("Wonder", undefined); // "Wonder Woman"  
let result3 = buildName("Bob", "Adams", "Sr."); // KO, troppi parametri  
let result4 = buildName("Bob", "Adams"); // "Bob Adams"
```

Parametri opzionali di una funzione

- Inoltre è possibile fornire valori di inizializzazione ai parametri
 - Usati se il parametro non viene fornito o se `undefined`
- Quelli con *inizializzazione* che vengono dopo tutti quelli richiesti sono trattati come *opzionali* (e quindi possono essere omessi in chiamata)
 - In questo caso, la funzione ha lo stesso tipo di quella con parametro opzionale

```
function buildName(firstName: string, lastName = "Woman") {  
  if (lastName) return firstName + " " + lastName;  
  else return firstName;  
}
```

tipo: (firstName: string, lastName?: string): string

Parametri opzionali di una funzione

- Quindi, se uno di questi parametri precede uno richiesto, deve sempre essere passato (eventualmente come `undefined`)

```
function buildName(firstName = "Will", lastName: string) {  
  return firstName + " " + lastName;  
}
```

```
let result1 = buildName("Bob"); // KO, pochi parametri  
let result2 = buildName("Bob", "Adams", "Sr."); // KO, troppi parametri  
let result3 = buildName("Bob", "Adams"); // OK -> "Bob Adams"  
let result4 = buildName(undefined, "Adams"); // OK -> "Will Adams"
```

tipo: ????

Parametri opzionali di una funzione

- Quindi, se uno di questi parametri precede uno richiesto, deve sempre essere passato (eventualmente come `undefined`)

```
function buildName(firstName = "Will", lastName: string) {  
  return firstName + " " + lastName;  
}
```

```
let result1 = buildName("Bob"); // KO, pochi parametri  
let result2 = buildName("Bob", "Adams", "Sr."); // KO, troppi parametri  
let result3 = buildName("Bob", "Adams"); // OK -> "Bob Adams"  
let result4 = buildName(undefined, "Adams"); // OK -> "Will Adams"
```

tipo: (firstName: string | undefined, lastName: string): string

Q & A